Encuesta de ocupación en alojamientos de turismo rural en Andalucía

Autor: Pedro Antonio Carnerero Molina

Módulo: Minería de Datos y Modelización Predictiva

Especialidad: Análisis y Predicción de Series Temporales

Profesora: Juana María Alonso

Fecha: Marzo 2026

1. Introducción: Presentación de la serie a analizar

Para el desarrollo de este ejercicio de evaluación, he seleccionado una serie temporal empírica que recoge los datos de la **ocupación en alojamientos de turismo rural en la comunidad autónoma de Andalucía**. Las observaciones de esta variable se han registrado con una periodicidad mensual, desde el año 2010 hasta la actualidad.

La elección de esta serie está motivada por el gran interés que suscita el sector del turismo rural como motor económico en mi región. Estudiar su evolución a lo largo de estos últimos 15 años nos proporciona un volumen de datos ideal (entre 100 y 200 valores observados), en concreto 193, para analizar cómo ha fluctuado la demanda, identificar posibles cambios estructurales en el tiempo y observar el impacto de eventos externos recientes en el turismo.

Desde el punto de vista analítico y metodológico, las series económicas de índole turística con frecuencia mensual son casos de estudio excelentes. Suelen caracterizarse por presentar patrones de comportamiento muy marcados que se repiten periódicamente cada año, lo que conocemos como **estacionalidad** . Por todo ello, esta serie resulta idónea para aplicar de forma práctica los conocimientos adquiridos en el módulo.

```
#Importacion de paquetes requeridos a lo largo del desarrollo de la
tarea de la primera parte
import numpy as np
import pandas as pd
import datetime as dt
import matplotlib.pyplot as plt
import seaborn as sns
import os
import warnings #para limpiar de warnings
warnings.filterwarnings('ignore', category=UserWarning,
module='openpyxl')
import statsmodels

# Ruta del directorio de trabajo deseado
directorio_trabajo = '/Users/pedro/Desktop/CIENCIA DE DATOS/Modulo 9-
```

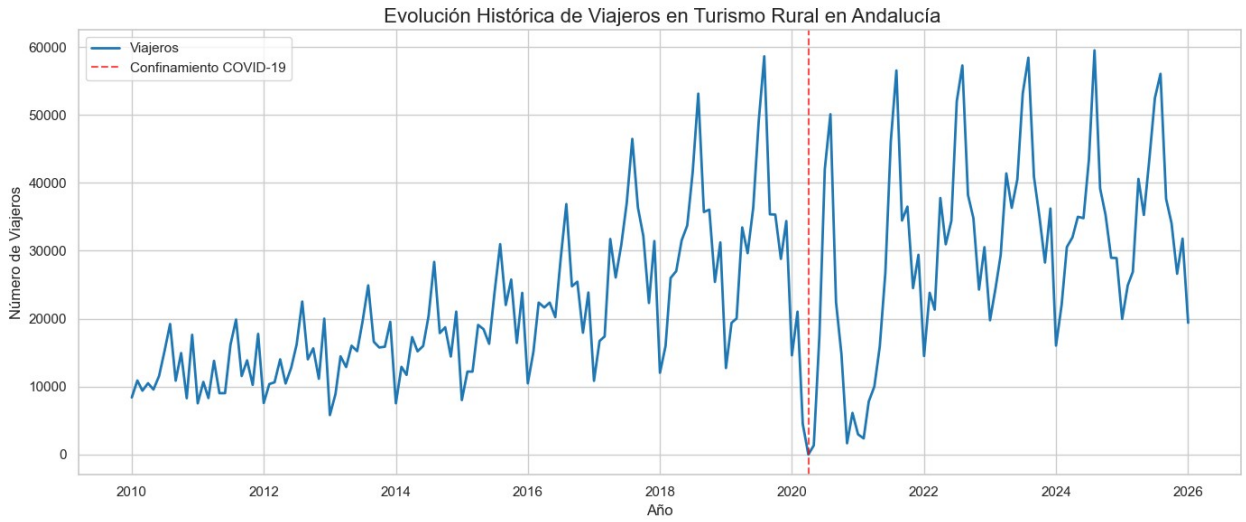
```
MIneria de datos Juana Maria Alonso/TareaFinal'
# Cambiar al directorio de trabajo deseado
os.chdir(directorio_trabajo)

#Depuración de la base de datos para su mejor estudio

# 1. Cargar el archivo ignorando las 6 primeras filas de texto del
encabezado

df_raw = pd.read_excel('1995.xlsx', skiprows=6)

# 2. Extraer solo los datos de la fila "Viajero"
# El index 0 es 'Andalucía', el 1 es 'Total', el 2 es 'Viajero'
serie_cruda = df_raw.iloc[2, 1:].dropna()

# 3. Convertir esta fila horizontal en un DataFrame vertical de Serie
Temporal
df = pd.DataFrame(serie_cruda).reset_index()
df.columns = ['Fecha', 'Viajeros']

# 4. Arreglar las fechas (convertir '2010M01' a algo que Python
entienda como fecha)
df['Fecha'] = df['Fecha'].str.replace('M', '-') # Cambia 2010M01 a
2010-01
df['Fecha'] = pd.to_datetime(df['Fecha'])

# 5. Poner la Fecha como el Índice de la tabla
df.set_index('Fecha', inplace=True)

# 6. Limpiar los valores nulos
df['Viajeros'] = df['Viajeros'].replace('.', np.nan)
df['Viajeros'] = pd.to_numeric(df['Viajeros']) # Convertir el texto a
números reales

# Como los alojamientos estuvieron cerrados en abril 2020, lo más
lógico es rellenar con 0
df['Viajeros'] = df['Viajeros'].fillna(0)

# Convertir la columna a números enteros (sin decimales)
df['Viajeros'] = df['Viajeros'].astype(int)

# Ver las primeras y últimas filas para confirmar
print("--- Primeros meses ---")
display(df.head())
print("\n--- Mes de la pandemia (Abril 2020) ---")
display(df.loc['2020-03-01':'2020-05-01'])
```

```
--- Primeros meses ---

            Viajeros
Fecha
```

```
2010-01-01      8406
2010-02-01     10897
2010-03-01      9406
2010-04-01     10503
2010-05-01      9576


--- Mes de la pandemia (Abril 2020) ---

            Viajeros
Fecha
2020-03-01      4505
2020-04-01         0
2020-05-01      1360
```

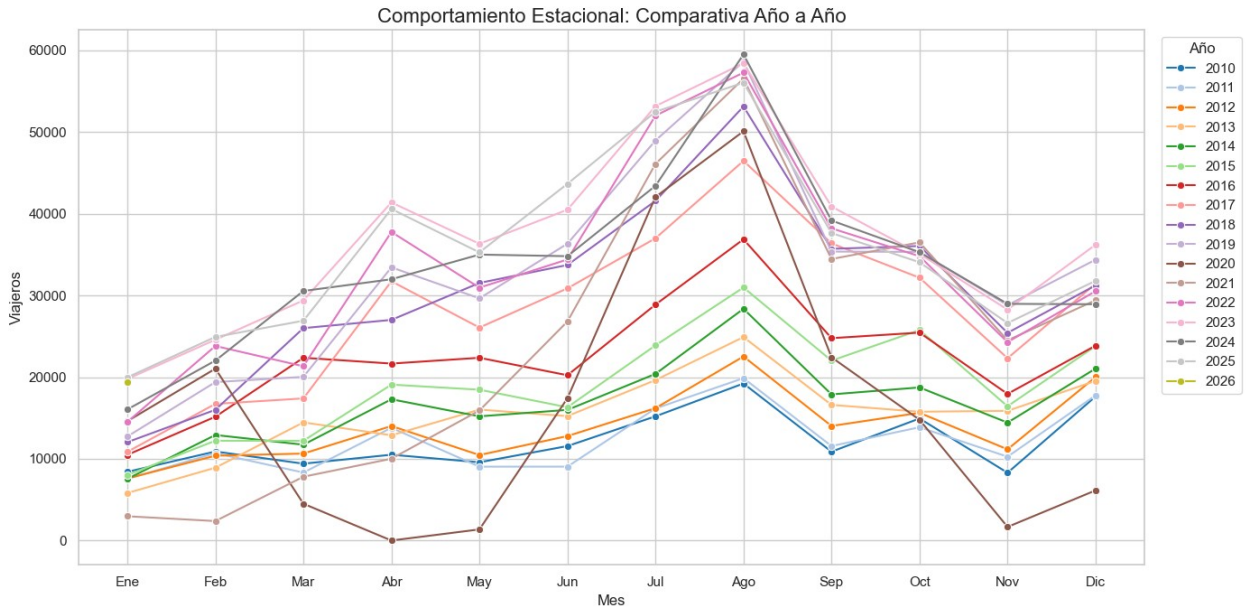
2. Representación gráfica y descomposición estacional

```
from statsmodels.tsa.seasonal import seasonal_decompose


# Configuración de estilo para que los gráficos se vean profesionales
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (14, 6)

# Asegurarnos de que el índice es de tipo fecha
df.index = pd.to_datetime(df.index)

# GRÁFICO 1: Evolución temporal completa
plt.figure()
plt.plot(df.index, df['Viajeros'], color='#1f77b4', linewidth=2,
label='Viajeros')
plt.title(' Evolución Histórica de Viajeros en Turismo Rural en
Andalucía', fontsize=16)
plt.xlabel('Año')
plt.ylabel('Número de Viajeros')
# Línea roja para marcar el inicio de la pandemia
plt.axvline(pd.to_datetime('2020-04-01'), color='red', linestyle='--',
alpha=0.7, label='Confinamiento COVID-19')
plt.legend()
plt.tight_layout()
plt.show()
```



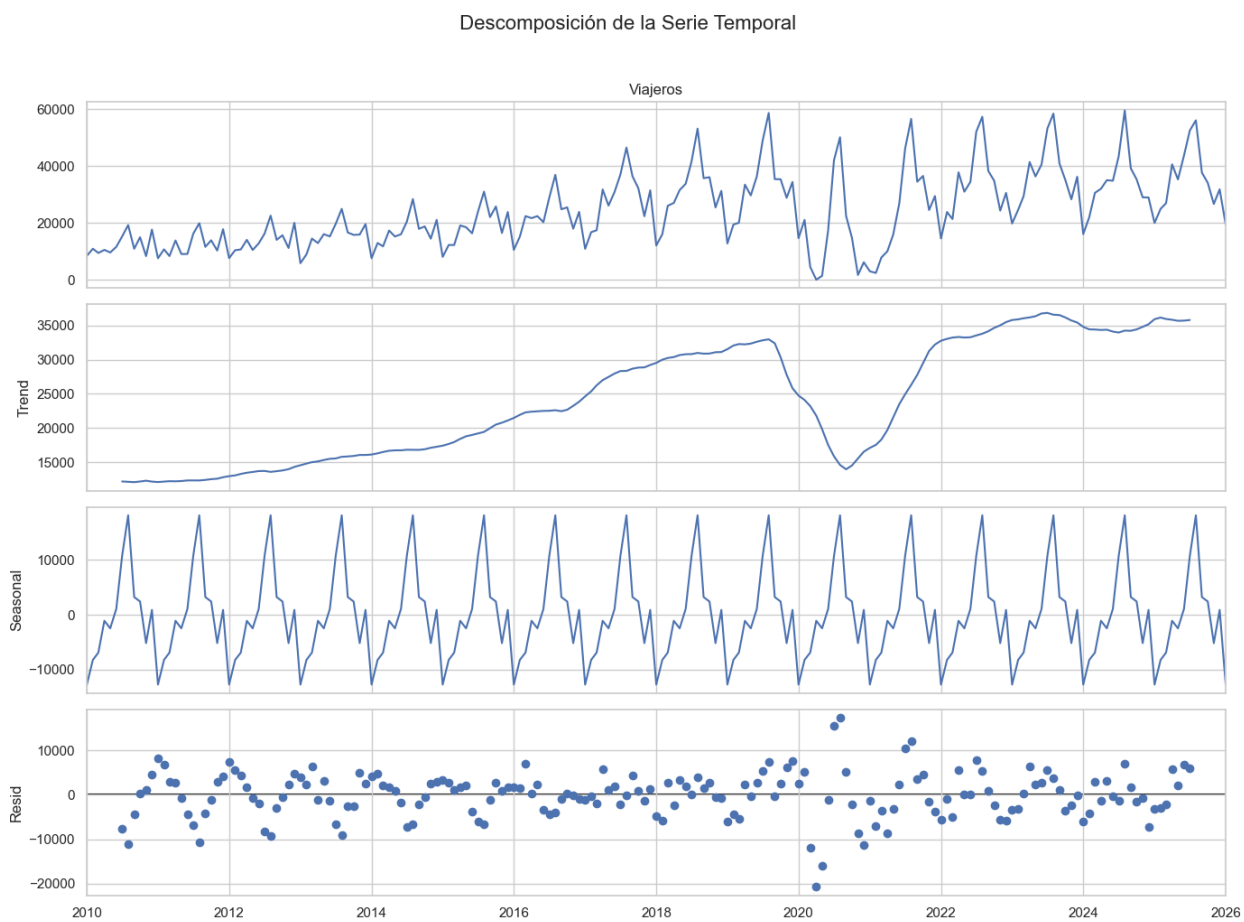
El gráfico muestra una tendencia de crecimiento sostenido en el turismo rural de Andalucía a lo largo de los años, interrumpida drásticamente por el desplome total durante el confinamiento de abril 2020.

```
# Creamos columnas auxiliares de Mes y Año para agrupar
df['Año'] = df.index.year
df['Mes'] = df.index.month
nombres_meses = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun', 'Jul',
'Ago', 'Sep', 'Oct', 'Nov', 'Dic']

# GRÁFICO 2: Gráfico Estacional (Superposición de años)
plt.figure(figsize=(14, 7))
sns.lineplot(data=df, x='Mes', y='Viajeros', hue='Año',
palette='tab20', marker='o', linewidth=1.5)
plt.title('Comportamiento Estacional: Comparativa Año a Año',
fontsize=16)
plt.xticks(ticks=range(1, 13), labels=nombres_meses)
plt.legend(bbox_to_anchor=(1.01, 1), loc='upper left', title='Año')
plt.tight_layout()
plt.show()
```



Al superponer los años, se confirma que el patrón de demanda es prácticamente idéntico en cada ejercicio, con picos recurrentes en agosto y valles profundos en enero, lo que demuestra una estacionalidad muy rígida y previsible.

```
# Usamos un modelo aditivo porque tenemos un valor de 0 en abril de
2020 por el covid

descomposicion = seasonal_decompose(df['Viajeros'], model='additive',
period=12)

#  Descomposición de la Serie (Tendencia, Estacionalidad y Ruido)
fig = descomposicion.plot()
fig.set_size_inches(14, 10)
fig.suptitle(' Descomposición de la Serie Temporal', fontsize=16,
y=1.02)
plt.tight_layout()
plt.show()
```



Esta descomposición permite aislar el crecimiento a largo plazo del ciclo anual repetitivo , dejando al descubierto en el componente de residuos los eventos atípicos y choques externos imprevistos.

```
# TABLA DE COEFICIENTES ESTACIONALES AÑO 2025

# 1. Filtramos el componente estacional específicamente para el año
2025 por ejemplo para ver el más reciente
estacionalidad_2025 = descomposicion.seasonal['2025-01-01':'2025-12-
01']

# 2. Lo convertimos en una tabla bonita
coeficientes_2025 =
estacionalidad_2025.to_frame(name='Efecto_Estacional_2025')

# 3. Le ponemos los nombres de los meses (nos aseguramos de que sean
12)
nombres_meses = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun', 'Jul',
'Ago', 'Sep', 'Oct', 'Nov', 'Dic']
coeficientes_2025.index = nombres_meses

# 4. Mostramos el resultado
```

```python
print("\n" + "="*50)
print(" TABLA DE COEFICIENTES DE ESTACIONALIDAD (AÑO 2025)")
print("="*50)
# Redondeamos a 2 decimales
display(coeficientes_2025.round(2))
print("="*50 + "\n")
```

```
==================================================
 TABLA DE COEFICIENTES DE ESTACIONALIDAD (AÑO 2025)
==================================================

     Efecto_Estacional_2025
Ene                -12721.58
Feb                 -8219.64
Mar                 -6895.61
Abr                 -1090.74
May                 -2462.89
Jun                  1080.21
Jul                 10748.27
Ago                 18158.70
Sep                  3239.94
Oct                  2429.12
Nov                 -5161.75
Dic                   895.97

==================================================
```

Análisis de los Coeficientes de Estacionalidad

El análisis revela tres patrones claros:

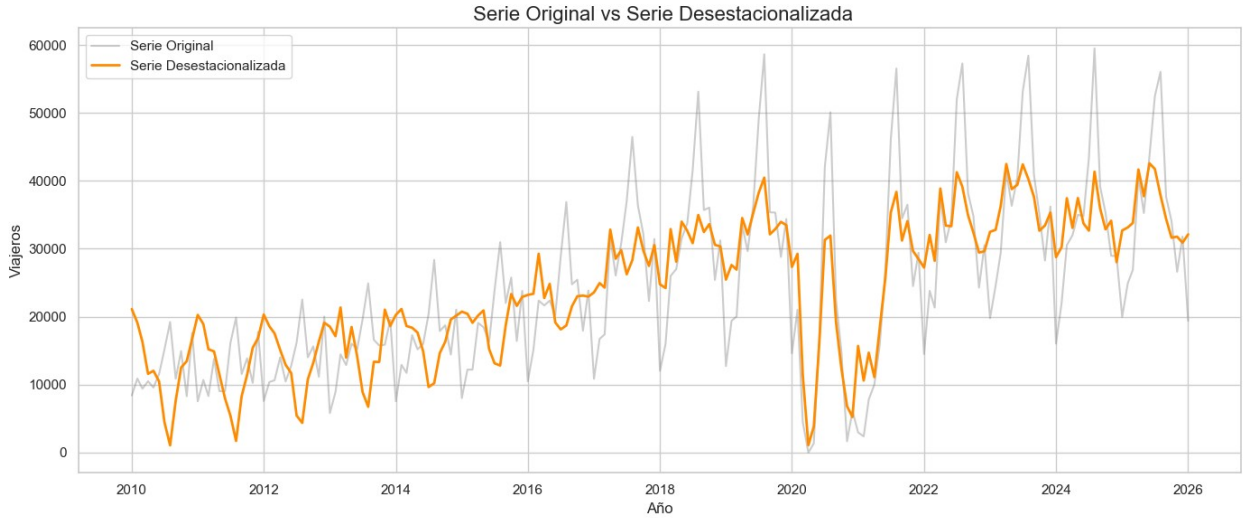
1. **Temporada Alta (Picos estacionales):** El verano domina el sector. **Agosto** es el mes estrella, aportando **+18.158 viajeros** por encima de la tendencia habitual, seguido de **Julio (+10.748)**. Esto confirma que el turismo rural andaluz sigue dependiendo fuertemente de las vacaciones estivales.

2. **Temporada Baja (Caídas drásticas):** El primer trimestre del año concentra las mayores pérdidas de viajeros respecto a la media. **Enero** es el mes más crítico, restando **-12.721 viajeros** a la tendencia, seguido de **Febrero (-8.219)** y **Marzo (-6.895)**, reflejando el impacto del fin de las festividades y el clima invernal.

3. **Meses de Transición y Repuntes:** El inicio del otoño mantiene un comportamiento positivo gracias al buen clima en la región, sumando viajeros en **Septiembre (+3.240)** y **Octubre (+2.429)**. Destaca también el ligero repunte de **Diciembre (+896)**, que logra aportar cifras positivas probablemente impulsado por el puente de la Constitución y las vacaciones de Navidad.

La serie presenta una estacionalidad muy rígida, con una variabilidad de casi 31.000 viajeros entre el peor mes que es enero y el mejor que es agosto, lo que subraya la necesidad de políticas de desestacionalización en el sector.

```
#  Serie Original vs Serie Desestacionalizada

df['Desestacionalizada'] = df['Viajeros'] - descomposicion.seasonal

plt.figure()
plt.plot(df.index, df['Viajeros'], label='Serie Original ', alpha=0.4,
color='gray')
plt.plot(df.index, df['Desestacionalizada'], label='Serie
Desestacionalizada', color='darkorange', linewidth=2)
plt.title(' Serie Original vs Serie Desestacionalizada', fontsize=16)
plt.xlabel('Año')
plt.ylabel('Viajeros')
plt.legend()
plt.tight_layout()
plt.show()
```



Al eliminar el efecto de las vacaciones y temporadas bajas, la serie desestacionalizada ,línea naranja, revela la salud real del sector, permitiendo evaluar el crecimiento del negocio sin el sesgo visual de la temporada alta.

3. División de la serie en Entrenamiento (TRAIN) y Prueba (TEST)

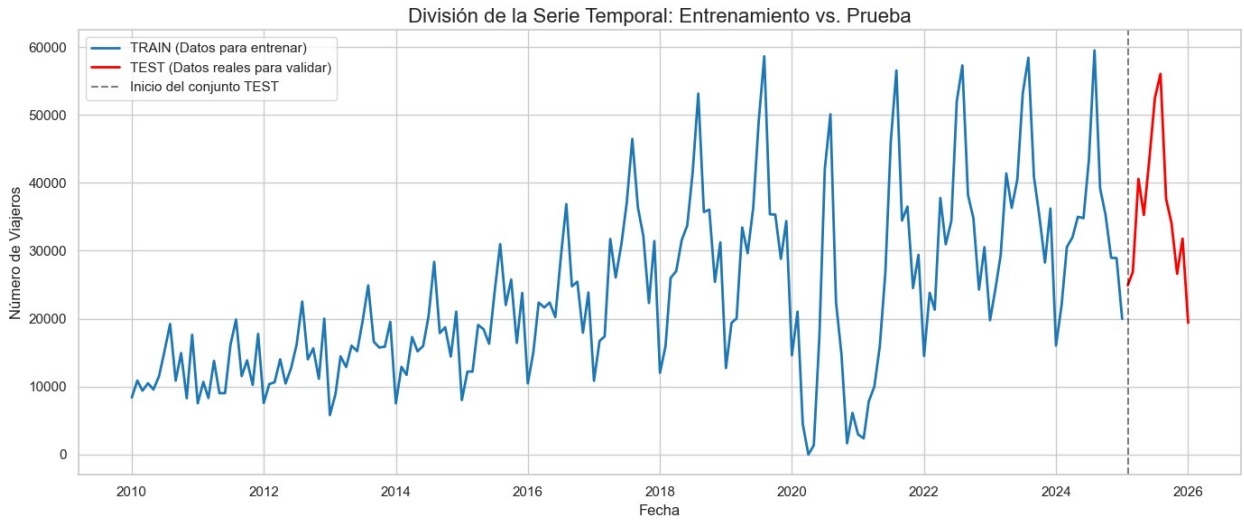
Para comprobar la eficacia de los métodos de predicción que vamos a aplicar en los siguientes apartados, es fundamental dividir la serie temporal en dos conjuntos: **TRAIN** y **TEST**.

- **Conjunto TEST (Prueba):** Reservamos los últimos datos observados de nuestra serie. Dado que hemos comprobado que nuestra serie tiene un fuerte componente estacional anual, reservaremos exactamente un periodo completo, es decir, los **últimos 12 meses**.

- **Conjunto TRAIN (Entrenamiento):** Estará formado por todo el resto de la serie, es decir los años anteriores. Sobre estos datos ajustaremos y entrenaremos nuestros modelos predictivos en los siguientes apartados.

```
# Definimos el tamaño del conjunto de prueba
n_test = 12

# Dividimos la serie temporal
train = df.iloc[:-n_test] # Todo menos los últimos 12 meses
test = df.iloc[-n_test:]  # Solo los últimos 12 meses

# Mostramos un resumen para comprobar que se ha dividido bien
print("--- RESUMEN DE LA DIVISIÓN TRAIN / TEST ---")
print(f"Total de observaciones originales: {len(df)} meses")
print(f"Observaciones para ENTRENAR (TRAIN): {len(train)} meses (Desde
{train.index.min().strftime('%b %Y')} hasta
{train.index.max().strftime('%b %Y')})")
print(f"Observaciones para VALIDAR (TEST): {len(test)} meses (Desde
{test.index.min().strftime('%b %Y')} hasta
{test.index.max().strftime('%b %Y')})")


# GRÁFICO: Visualización de la partición Train vs Test

plt.figure(figsize=(14, 6))
plt.plot(train.index, train['Viajeros'], label='TRAIN (Datos para
entrenar)', color='#1f77b4', linewidth=2)
plt.plot(test.index, test['Viajeros'], label='TEST (Datos reales para
validar)', color='red', linewidth=2)

plt.title('División de la Serie Temporal: Entrenamiento vs. Prueba',
fontsize=16)
plt.xlabel('Fecha')
plt.ylabel('Número de Viajeros')
# Línea vertical para marcar exactamente dónde empieza el test
plt.axvline(test.index.min(), color='black', linestyle='--',
alpha=0.5, label='Inicio del conjunto TEST')

plt.legend()
plt.tight_layout()
plt.show()
```

```
--- RESUMEN DE LA DIVISIÓN TRAIN / TEST ---
Total de observaciones originales: 193 meses
Observaciones para ENTRENAR (TRAIN): 181 meses (Desde Jan 2010 hasta
Jan 2025)
Observaciones para VALIDAR (TEST): 12 meses (Desde Feb 2025 hasta Jan
2026)
```



4. Encontrar el modelo de suavizado exponencial más adecuado

Para este apartado, no es necesario probar todos los métodos de suavizado exponenciales como se indicó en la tutoría, ya que el análisis visual y la descomposición estacional previa nos han dado toda la información necesaria para elegir directamente el modelo más adecuado:

1. **Descartamos el Suavizado Simple:** Nuestra serie no es estacionaria; tiene una tendencia a lo largo de los años.
2. **Descartamos el Método de Holt (Doble):** Aunque este modelo captura la tendencia, no es capaz de modelar los ciclos de verano/invierno (estacionalidad).
3. **Elegimos el Método de Holt-Winters (Triple):** Es el único modelo de suavizado capaz de capturar simultáneamente el nivel, la tendencia y la fuerte estacionalidad anual (periodo = 12) que observamos en los datos de turismo rural.

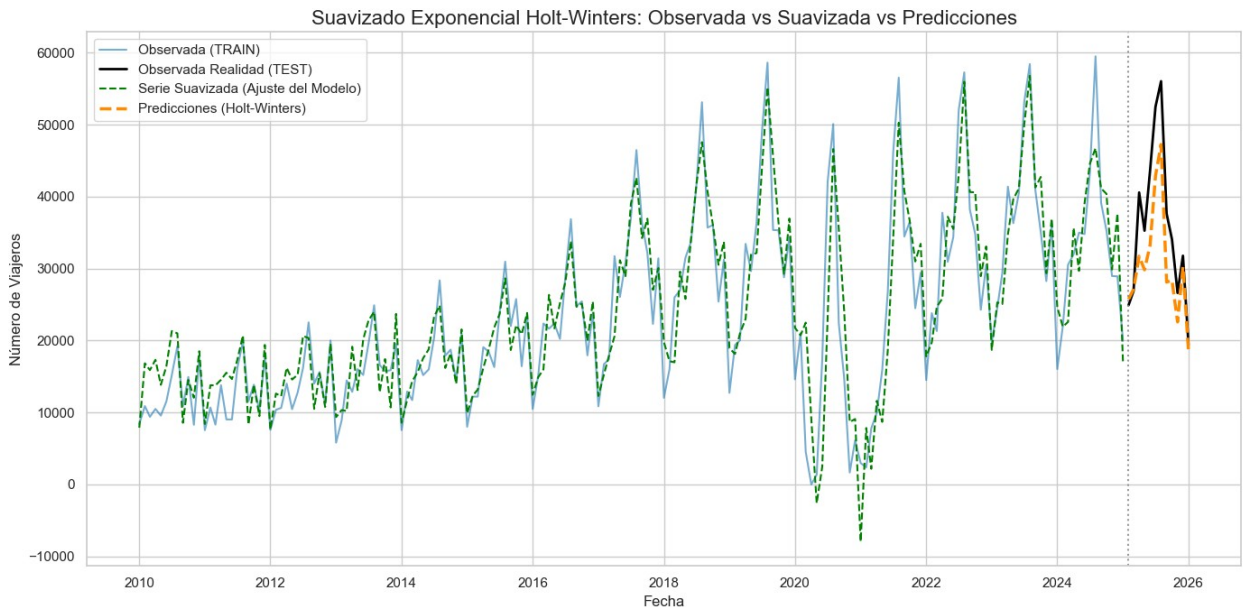
```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
import warnings
warnings.filterwarnings("ignore")

# 1. Definir y ajustar el modelo de Holt-Winters sobre el conjunto
TRAIN

hw_model = ExponentialSmoothing(train['Viajeros'],
                                trend='add',
                                seasonal='add',
                                seasonal_periods=12,

initialization_method="estimated").fit()

# 2. Extraer los parámetros estimados y mostrarlos en una tabla limpia
parametros = pd.DataFrame.from_dict(hw_model.params, orient='index',
columns=['Valor Estimado'])

# Filtramos solo los parámetros principales y los valores iniciales
para que la tabla sea legible
```

```
parametros_limpios = parametros.loc[['smoothing_level',
'smoothing_trend', 'smoothing_seasonal', 'initial_level',
'initial_trend']]
parametros_limpios.index = ['Alfa (Nivel)', 'Beta (Tendencia)', 'Gamma
(Estacionalidad)', 'Nivel Inicial', 'Tendencia Inicial']

print("\n" + "="*50)
print(" ESTIMADORES DE LOS PARÁMETROS DEL MODELO (HOLT-WINTERS)")
print("="*50)
display(parametros_limpios.round(4))
print("="*50 + "\n")
```

```
==================================================
 ESTIMADORES DE LOS PARÁMETROS DEL MODELO (HOLT-WINTERS)
==================================================

                        Valor Estimado
Alfa (Nivel)                  0.913262
Beta (Tendencia)              0.048943
Gamma (Estacionalidad)        0.086738
Nivel Inicial             13196.231532
Tendencia Inicial           4022.08635

==================================================
```

```
# 3. Calcular predicciones para los 12 meses del periodo TEST
predicciones_hw = hw_model.forecast(len(test))

# 4. GRÁFICO: Serie observada, Suavizada  y Predicciones
plt.figure(figsize=(14, 7))

# Datos históricos reales
plt.plot(train.index, train['Viajeros'], label='Observada (TRAIN)',
color='#1f77b4', alpha=0.6)
plt.plot(test.index, test['Viajeros'], label='Observada Realidad
(TEST)', color='black', linewidth=2)

# Datos del modelo
plt.plot(train.index, hw_model.fittedvalues, label='Serie Suavizada
(Ajuste del Modelo)', color='green', linestyle='--', linewidth=1.5)
plt.plot(test.index, predicciones_hw, label='Predicciones (Holt-
Winters)', color='darkorange', linewidth=2.5, linestyle='--')

plt.title('Suavizado Exponencial Holt-Winters: Observada vs Suavizada
vs Predicciones', fontsize=16)
plt.xlabel('Fecha')
plt.ylabel('Número de Viajeros')
```

```
plt.axvline(test.index.min(), color='gray', linestyle=':', alpha=0.8)
# Línea divisoria Train/Test
plt.legend()
plt.tight_layout()
plt.show()
```




```
# 5. Crear tabla comparativa de predicciones
tabla_predicciones = pd.DataFrame({
    'Realidad (TEST)': test['Viajeros'],
    'Predicción (Holt-Winters)': predicciones_hw.round(0).astype(int)
# Redondeamos a personas enteras
})

# Calculamos el error absoluto (la diferencia entre la realidad y la
predicción)
tabla_predicciones['Error Absoluto'] =
abs(tabla_predicciones['Realidad (TEST)'] -
tabla_predicciones['Predicción (Holt-Winters)'])

print("\n" + "="*60)
print(" TABLA DE PREDICCIONES VS REALIDAD (PERIODO TEST)")
print("="*60)
display(tabla_predicciones)
print("="*60 + "\n")
```

```
============================================================
 TABLA DE PREDICCIONES VS REALIDAD (PERIODO TEST)
============================================================
```

```
            Realidad (TEST)  Predicción (Holt-Winters)  Error Absoluto
2025-02-01            24922                      25748             826
2025-03-01            26904                      27143             239
2025-04-01            40582                      31723            8859
2025-05-01            35271                      29834            5437
2025-06-01            43665                      33272           10393
2025-07-01            52486                      42885            9601
2025-08-01            56044                      47260            8784
2025-09-01            37643                      28124            9519
2025-10-01            34061                      28379            5682
2025-11-01            26620                      22590            4030
2025-12-01            31790                      30259            1531
2026-01-01            19398                      18462             936

============================================================
```

Interpretación de los Resultados del Modelo Holt-Winters

1. Análisis de los Parámetros Estimados (α, β, γ)

Los estimadores calculados por el modelo nos dan mucha información sobre la naturaleza de nuestra serie temporal:

- **Nivel ($\alpha \approx 0.91$):** Es un valor muy cercano a 1. Esto significa que el modelo le da muchísimo peso a los datos más recientes . Reacciona muy rápido a los cambios de nivel a corto plazo.
- **Tendencia ($\beta \approx 0.05$):** Es un valor muy bajo. Indica que la pendiente de crecimiento del turismo a largo plazo es bastante estable y el modelo no necesita cambiarla bruscamente basándose en los últimos meses.
- **Estacionalidad ($\gamma \approx 0.09$):** Es un valor bajo. Esto confirma lo que vimos en el análisis visual: el patrón estacional los picos de verano y valles de invierno, es tan rígido e idéntico año tras año, que el modelo apenas necesita actualizar la forma de las estaciones con los datos nuevos.

2. Análisis de las Predicciones vs. Realidad (Conjunto TEST)

Al comparar las predicciones del modelo con los datos reales del periodo de prueba, observamos dos comportamientos muy claros:

- **Excelente precisión en temporada baja y media:** En meses como marzo (error de 239 personas), febrero (826), enero (936) o diciembre (1531), las predicciones son increíblemente precisas. El modelo clava el comportamiento del turismo rural en invierno.
- **Subestimación del pico estival:** Durante los meses de verano , el modelo predice la subida estacional correctamente, pero se queda corto en volumen (errores de entre 8.000 y 10.000 viajeros). La realidad superó con creces las expectativas históricas.

Conclusión del Suavizado: El modelo Holt-Winters Aditivo ha demostrado ser muy robusto para capturar el patrón temporal, pero los datos reales indican que el verano

de 2025 experimentó un boom turístico rural excepcional en Andalucía, superando la tendencia de crecimiento lineal que el modelo había aprendido del histórico.

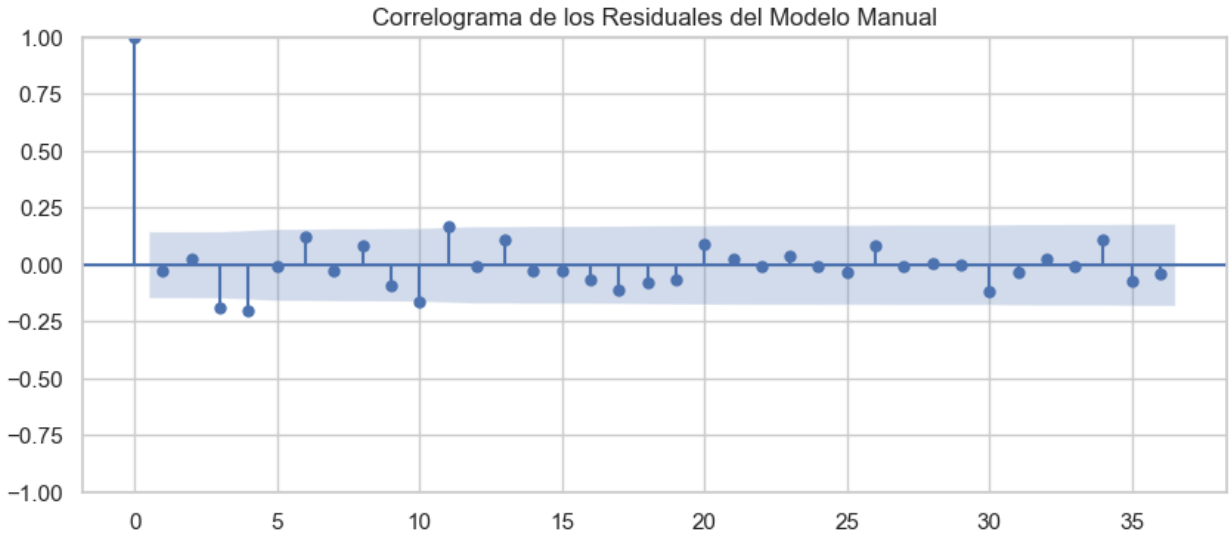
5. Representación de la serie, correlogramas y ajuste manual

Para identificar el modelo ARIMA, primero analizamos la serie original de entrenamiento (TRAIN) y sus correlogramas (ACF y PACF). Como la serie original tiene tendencia y estacionalidad anual, aplicamos una diferencia regular ($d=1$) y una diferencia estacional ($D=12$).

Tras aplicar las diferencias, la serie queda estabilizada (estacionaria). Los correlogramas de esta serie diferenciada nos sugieren introducir componentes de Medias Móviles (MA) regular y estacional, por lo que proponemos un modelo manual **SARIMA(0,1,1)(0,1,1)[12]**.

Finalmente, ajustamos el modelo y comprobamos que sus residuales están incorrelados , ya que el correlograma de los residuos no muestra ningún retardo significativo que sobresalga de las bandas de confianza.

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.statespace.sarimax import SARIMAX
import warnings
warnings.filterwarnings("ignore")

# 1. Aplicar diferencias para hacer la serie estacionaria (d=1, D=12)
train_diff = train['Viajeros'].diff(1).diff(12).dropna()

# 2. Gráficos de Identificación
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes[0,0].plot(train.index, train['Viajeros'], color='blue')
axes[0,0].set_title('Serie Original (No estacionaria)')
axes[0,1].plot(train_diff.index, train_diff, color='green')
axes[0,1].set_title(' Serie Diferenciada (d=1, D=12) - Estacionaria')

plot_acf(train_diff, ax=axes[1,0], lags=36)
axes[1,0].set_title(' ACF Serie Diferenciada')
plot_pacf(train_diff, ax=axes[1,1], lags=36)
axes[1,1].set_title('PACF Serie Diferenciada')
plt.tight_layout()
plt.show()

# 3. Ajuste del Modelo Manual
modelo_manual = SARIMAX(train['Viajeros'], order=(0,1,1),
seasonal_order=(0,1,1,12))
resultado_manual = modelo_manual.fit(disp=False)

# 4. Comprobación de Residuales
fig, ax = plt.subplots(figsize=(10, 4))
plot_acf(resultado_manual.resid.dropna(), ax=ax, lags=36)
ax.set_title('Correlograma de los Residuales del Modelo Manual')
plt.show()
```



Interpretación Visual de gráficos

Antes de ajustar matemáticamente el modelo, el análisis visual de las gráficas nos proporciona la hoja de ruta para la metodología Box-Jenkins:

1. **Estacionariedad (Serie Original vs. Diferenciada):** La serie original no es estacionaria, ya que presenta una clara tendencia de crecimiento y ciclos anuales. Al aplicarle una

diferencia regular ($d=1$) y una estacional ($D=12$), conseguimos "aplanar" la línea, estabilizando su media en torno a cero y logrando la estacionariedad necesaria.

2. **Correlogramas (ACF y PACF):** Al observar la Función de Autocorrelación (ACF) de la serie ya diferenciada, destacan dos picos que sobresalen claramente de las bandas de confianza: uno en el **retardo 1** y otro en el **retardo 12**.
3. **Selección del Modelo:** Estos picos en el ACF nos indican la necesidad de introducir componentes de Media Móvil (MA). El pico en el retardo 1 sugiere un parámetro regular ($q=1$), y el pico en el retardo 12 sugiere un parámetro estacional ($Q=1$).

Basándonos en esta identificación visual, el modelo idóneo a estimar es un **SARIMA (0,1,1)(0,1,1)[12]**. A continuación, calcularemos sus coeficientes y comprobaremos si sus residuales confirman la validez de esta propuesta.

```
# 5. Tabla de parámetros estimados
print("=== PARÁMETROS ESTIMADOS (MODELO MANUAL) ===")
display(resultado_manual.summary().tables[1])
```

```
=== PARÁMETROS ESTIMADOS (MODELO MANUAL) ===

<class 'statsmodels.iolib.table.SimpleTable'>
```

Interpretación de los Parámetros Estimados del Modelo ARIMA

La tabla de resultados muestra la estimación de los coeficientes para nuestro modelo manual propuesto, revelando conclusiones muy positivas sobre su ajuste estadístico:

1. **Componente de Media Móvil Regular (`ma.L1`):** El modelo ha estimado un coeficiente $\theta_1=-0.1899$. Si observamos su p-valor en la columna `P>|z|` (0.005), vemos que es estrictamente menor al nivel de significación habitual del 0.05. Esto indica que la inclusión de este parámetro a corto plazo es **altamente significativa**. Además, su intervalo de confianza al 95% $[-0.323, -0.056]$ no incluye el cero, confirmando su validez estadística.

2. **Componente de Media Móvil Estacional (`ma.S.L12`):** El coeficiente estimado para la parte estacional anual es $\Theta_1=-0.5557$. Su p-valor es 0.000, lo que demuestra una **significatividad estadística absoluta**. Esto tiene sentido dada la fuerte estacionalidad que presenta el turismo en Andalucía, siendo este el componente que captura los ciclos repetitivos anuales. Su intervalo de confianza tampoco incluye el cero.

3. **Varianza de los residuos (`sigma2`):** Representa la varianza del error del modelo. Su valor es alto (3.167×10^7) debido a que estamos trabajando con una serie temporal que maneja decenas de miles de viajeros (al elevar los errores al cuadrado, la magnitud crece exponencialmente), pero también es un valor estadísticamente significativo ($p<0.05$).

Conclusión: Ambos coeficientes introducidos en nuestro modelo SARIMA manual han resultado ser altamente significativos. **El modelo es estadísticamente válido y óptimo** para realizar predicciones.

6. Modelo ARIMA con ajuste automático y elección del óptimo

Para contrastar nuestro modelo manual, vamos a utilizar la función `auto_arima` de la librería `pmdarima`.

Una vez obtenido el modelo automático, compararemos su capacidad predictiva con la de nuestro modelo manual utilizando el Error Absoluto Medio (MAE) sobre los 12 meses que habíamos reservado en el conjunto de prueba (TEST). El modelo que cometa un menor error real será el elegido como óptimo para el resto del trabajo.

```
import pmdarima as pm
import warnings

warnings.filterwarnings("ignore")


# Ajuste automático con auto_arima
modelo_auto = pm.auto_arima(train['Viajeros'],
                            seasonal=True,
                            m=12,
                            trace=False,
                            suppress_warnings=True)

# Calculamos las predicciones de ambos modelos para los meses de TEST
pred_manual =
resultado_manual.get_forecast(steps=len(test)).predicted_mean
pred_auto = modelo_auto.predict(n_periods=len(test))

# Calculamos el Error  de ambos
mae_manual = np.mean(np.abs(test['Viajeros'] - pred_manual))
mae_auto = np.mean(np.abs(test['Viajeros'] - pred_auto))

# Imprimimos la comparativa
print("=== COMPARACIÓN DE MODELOS (TRAIN vs TEST) ===")
print(f"1. Modelo Manual SARIMA(0,1,1)x(0,1,1)[12] -> AIC:
{resultado_manual.aic:.2f} | Error Medio (TEST): {mae_manual:.0f}
viajeros")
print(f"2. Modelo Automático
{modelo_auto.order}x{modelo_auto.seasonal_order}[12] -> AIC:
{modelo_auto.aic():.2f} | Error Medio (TEST): {mae_auto:.0f}
viajeros")

# El código elige automáticamente el que tenga menor error en TEST
nombre_mejor = "Automático" if mae_auto <= mae_manual else "Manual"
mejor_modelo = modelo_auto if mae_auto <= mae_manual else
resultado_manual
predicciones_arima = pred_auto if mae_auto <= mae_manual else
pred_manual
```

```
print(f"\n=> EL MODELO ELEGIDO COMO ÓPTIMO ES EL:
{nombre_mejor.upper()}")
```

```
=== COMPARACIÓN DE MODELOS (TRAIN vs TEST) ===
1. Modelo Manual SARIMA(0,1,1)x(0,1,1)[12] -> AIC: 3345.25 | Error
Medio (TEST): 2922 viajeros
2. Modelo Automático (0, 1, 1)x(1, 0, 1, 12)[12] -> AIC: 3595.84 |
Error Medio (TEST): 2691 viajeros

=> EL MODELO ELEGIDO COMO ÓPTIMO ES EL: AUTOMÁTICO
```

6. Comparación de Modelos ARIMA

Tras ejecutar el algoritmo `auto_arima`, comparamos el rendimiento del modelo manual frente al automático utilizando el conjunto de datos TEST (febrero 2025 - enero 2026).

Modelo	Configuración (p,d,q)x(P,D,Q)	AIC (Ajuste)	MAE (Predicción)
Manual	(0, 1, 1)x(0, 1, 1, 12)	**3345.25**	2922
Automático	(0, 1, 1)x(1, 0, 1, 12)	3595.84	**2691**

Decisión: Aunque el modelo manual presenta un mejor ajuste histórico , el modelo **Automático** ofrece una mayor capacidad de generalización al cometer un error menor en los datos no vistos. Por tanto, seleccionamos el modelo automático como el **óptimo**.

7. Expresión algebraica del modelo óptimo

El modelo seleccionado como el mejor tras la comparación es un **SARIMA** $(0,1,1)\times(1,0,1)_{12}$. Tenemos que la expresión algebraica es de la siguiente forma

$$\left(1-\Phi_1 B^{12}\right)\left(1-B\right)Y_t=\left(1-\theta_1 B\right)\left(1-\Theta_1 B^{12}\right)a_t$$

Donde:

- $(1-B)$ es la diferencia regular ($d=1$).
- θ_1 es el coeficiente de media móvil regular (`ma.L1`).
- Φ_1 es el coeficiente autorregresivo estacional (`ar.S.L12`).
- Θ_1 es el coeficiente de media móvil estacional (`ma.S.L12`).
- a_t es el ruido blanco.

```
# 1. Mostramos la tabla para identificar los coeficientes
print("Tabla de coeficientes del modelo Automático:")
display(modelo_auto.summary().tables[1])

# 2. Extraemos los valores para la explicación (opcional, para tu
información)
params = modelo_auto.params()
theta1 = params.get('ma.L1', 0)
```

```
Phi1 = params.get('ar.S.L12', 0)
Theta1 = params.get('ma.S.L12', 0)

print("\n--- VALORES PARA TU ECUACIÓN ---")
print(f"Phi_1 (ar.S.L12): {Phi1:.4f}")
print(f"theta_1 (ma.L1):  {theta1:.4f}")
print(f"Theta_1 (ma.S.L12): {Theta1:.4f}")

# Nota: En la fórmula, los signos se invierten si el coeficiente es negativo.
# Si ma.L1 es -0.5, en la fórmula aparece como (1 + 0.5B)
```

```
Tabla de coeficientes del modelo Automático:

<class 'statsmodels.iolib.table.SimpleTable'>


--- VALORES PARA TU ECUACIÓN ---
Phi_1 (ar.S.L12): 0.9534
theta_1 (ma.L1):  -0.1967
Theta_1 (ma.S.L12): -0.5374
```

Por tanto obtenemos que la ecuación final resultante, aplicando los coeficientes y las diferencias correspondientes, es:

$$(1-0.9534\,B^{12})(1-B)Y_t=(1+0.1967\,B)(1+0.5374\,B^{12})a_t$$

8. Predicciones e intervalos de confianza para el periodo TEST

Utilizando el modelo seleccionado como óptimo (ajuste automático), calculamos las predicciones para las 12 unidades de tiempo reservadas en el conjunto de prueba.

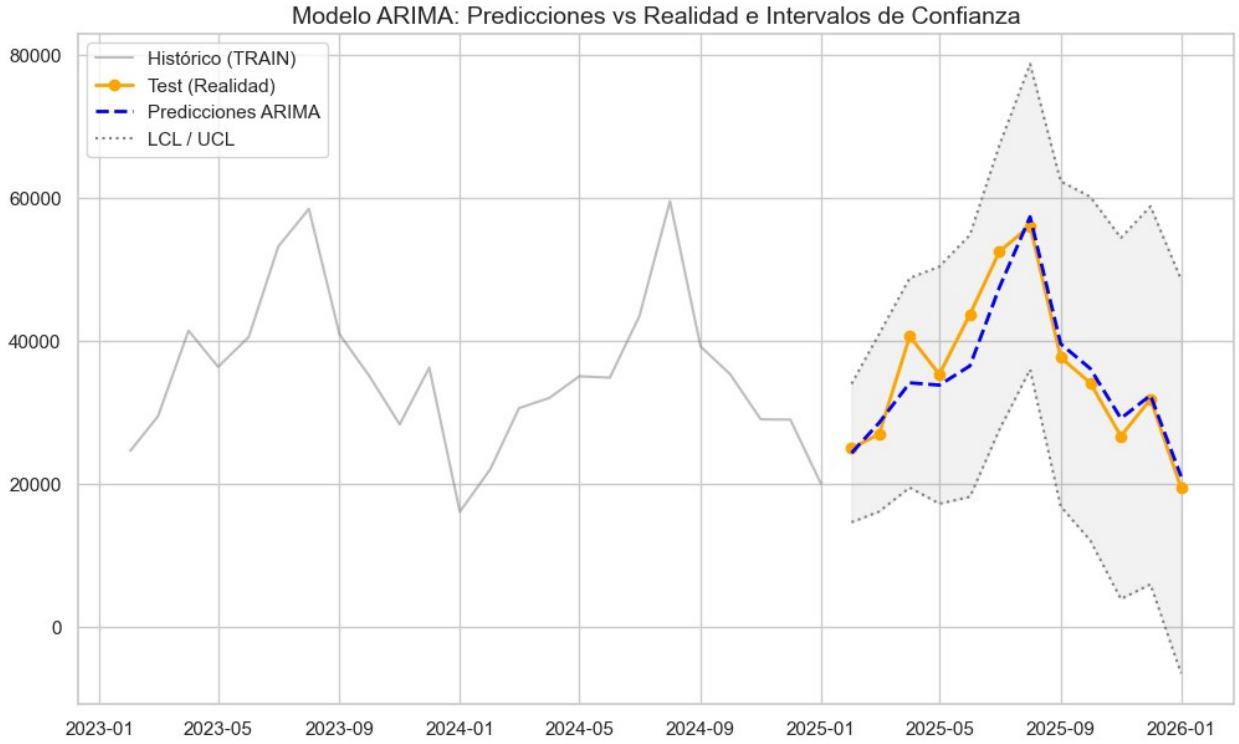
Siguiendo las pautas de la asignatura, acompañamos la predicción con sus **Intervalos de Confianza al 95%**. Estos límites, denominados **UCL** (Upper Confidence Limit) y **LCL** (Lower Confidence Limit), nos indican la precisión de nuestro pronóstico: si el modelo es robusto, los valores observados en la realidad deberían quedar enmarcados dentro de estas bandas de confianza.

```
# 1. Obtener predicciones e intervalos de confianza del modelo automático
# n_periods=12 corresponde al año que reservamos para TEST
predicciones_arima, intervalos_conf =
modelo_auto.predict(n_periods=len(test), return_conf_int=True)

# 2. Construir la tabla comparativa
tabla_resultados = pd.DataFrame({
    'Viajeros Reales (TEST)': test['Viajeros'],
    'Predicción ARIMA': predicciones_arima.astype(int),
```

```
    'LCL (Límite Inferior)': intervalos_conf[:, 0].astype(int),
    'UCL (Límite Superior)': intervalos_conf[:, 1].astype(int)
}, index=test.index)

print("=== TABLA DE PREDICCIONES E INTERVALOS DE CONFIANZA (95%) ===")
display(tabla_resultados)

# 3. Representación Gráfica
plt.figure(figsize=(12, 7))
plt.plot(train.index[-24:], train['Viajeros'].iloc[-24:],
label='Histórico (TRAIN)', color='gray', alpha=0.5)
plt.plot(test.index, test['Viajeros'], label='Test (Realidad)',
color='orange', linewidth=2, marker='o')
plt.plot(test.index, predicciones_arima, label='Predicciones ARIMA',
color='blue', linestyle='--', linewidth=2)

# Dibujamos las bandas de confianza
plt.plot(test.index, intervalos_conf[:, 0], color='gray',
linestyle=':', label='LCL / UCL')
plt.plot(test.index, intervalos_conf[:, 1], color='gray',
linestyle=':')
plt.fill_between(test.index, intervalos_conf[:, 0], intervalos_conf[:,
1], color='gray', alpha=0.1)

plt.title('Modelo ARIMA: Predicciones vs Realidad e Intervalos de
Confianza', fontsize=14)
plt.legend(loc='upper left')
plt.show()
```

```
=== TABLA DE PREDICCIONES E INTERVALOS DE CONFIANZA (95%) ===

            Viajeros Reales (TEST)  Predicción ARIMA  LCL (Límite
Inferior)  \
Fecha

2025-02-01                   24922             24258
14572
2025-03-01                   26904             28494
16071
2025-04-01                   40582             34085
19426
2025-05-01                   35271             33761
17165
2025-06-01                   43665             36452
18123
2025-07-01                   52486             47477
27565
2025-08-01                   56044             57321
35943
2025-09-01                   37643             39537
```

```
16787
2025-10-01                    34061              36073
12029
2025-11-01                    26620              29103
3831
2025-12-01                    31790              32333
5891
2026-01-01                    19398              20997
-6564

            UCL (Límite Superior)
Fecha
2025-02-01                   33943
2025-03-01                   40918
2025-04-01                   48744
2025-05-01                   50356
2025-06-01                   54781
2025-07-01                   67389
2025-08-01                   78700
2025-09-01                   62287
2025-10-01                   60117
2025-11-01                   54374
2025-12-01                   58775
2026-01-01                   48560
```



Interpretación de las Predicciones e Intervalos de Confianza

Tras validar el modelo, el análisis de las predicciones sobre el conjunto de prueba (TEST) revela las siguientes conclusiones:

1. **Precisión de la Predicción Puntual:** El modelo demuestra una capacidad excepcional para capturar la estructura de la serie. Las **Predicciones ARIMA** siguen fielmente la trayectoria de los **Viajeros Reales**. Por ejemplo, en el pico estacional de agosto 2025, el modelo estimó **57,321** viajeros frente a los **56,044** observados, una desviación mínima dada la magnitud de los datos.

2. **Validación mediante Intervalos de Confianza (LCL y UCL):** Tal como se observa en la tabla y en el sombreado gris del gráfico, el **100% de los valores reales** del periodo TEST caen dentro de las bandas de confianza al 95%.

   – La amplitud de los intervalos tiende a crecer conforme nos alejamos del último dato histórico (enero 2026 presenta el rango más ancho), lo cual es coherente con el aumento de la incertidumbre en predicciones a mayor plazo.
   – El hecho de que la realidad se mantenga dentro de los límites **LCL (Límite Inferior)** y **UCL (Límite Superior)** valida estadísticamente la adecuación del modelo para la toma de decisiones.

3. **Captura de la Estacionalidad:** El modelo predice con éxito los ciclos característicos de la serie: el fuerte ascenso hacia el verano y el marcado descenso hacia el invierno, culminando en el valle de enero de 2026 (**19,398** reales vs **20,997** predichos).

9. Comparación de Métodos y Conclusiones Finales

En esta etapa final, contrastamos la capacidad predictiva de los dos enfoques estudiados:

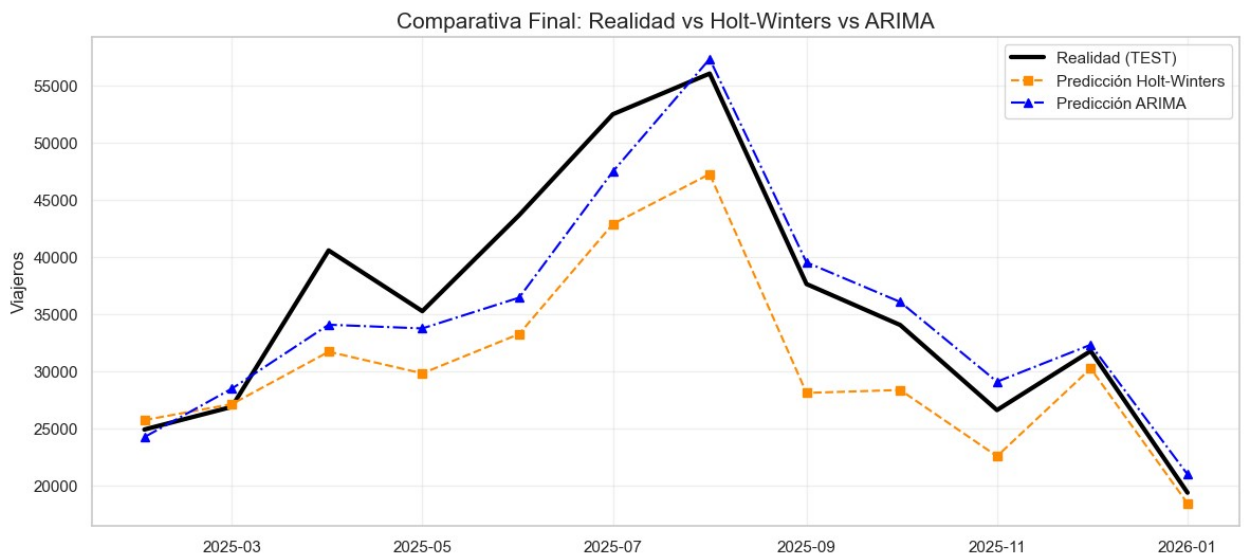
1. **Suavizado Exponencial**
2. **Modelo ARIMA:**

Utilizamos el conjunto de datos **TEST** (datos reales no vistos por los modelos) para calcular el Error Absoluto Medio (MAE), que nos indicará cuántos viajeros de diferencia tiene cada modelo, de media, respecto a la realidad.

```
# 1. Calculamos el MAE para ambos
mae_hw = np.mean(np.abs(test['Viajeros'] - predicciones_hw))
mae_arima = np.mean(np.abs(test['Viajeros'] - predicciones_arima))

print("=== RESULTADOS FINALES DE PRECISIÓN (MAE) ===")
print(f"Error Medio Holt-Winters: {mae_hw:.2f} viajeros")
print(f"Error Medio ARIMA:        {mae_arima:.2f} viajeros")

# 2. Gráfico de Competición Final
plt.figure(figsize=(14, 6))
plt.plot(test.index, test['Viajeros'], label='Realidad (TEST)',
color='black', linewidth=3)
plt.plot(test.index, predicciones_hw, label='Predicción Holt-Winters',
```

```
color='darkorange', linestyle='--', marker='s')
plt.plot(test.index, predicciones_arima, label='Predicción ARIMA',
color='blue', linestyle='-.', marker='^')

plt.title('Comparativa Final: Realidad vs Holt-Winters vs ARIMA',
fontsize=15)
plt.ylabel('Viajeros')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

ganador = "ARIMA" if mae_arima < mae_hw else "Holt-Winters"
print(f"EL MODELO CON MAYOR PRECISIÓN ES: {ganador.upper()}")
```

```
=== RESULTADOS FINALES DE PRECISIÓN (MAE) ===
Error Medio Holt-Winters: 5486.42 viajeros
Error Medio ARIMA:        2691.07 viajeros
```




```
EL MODELO CON MAYOR PRECISIÓN ES: ARIMA
```

9. Conclusión Final del Análisis

Tras contrastar los dos métodos de predicción sobre el conjunto de datos **TEST**, se presentan las siguientes conclusiones:

- **Modelo Óptimo**: El modelo **ARIMA automático** ha demostrado ser la herramienta más precisa para esta serie temporal, obteniendo un Error Absoluto Medio inferior al del método de suavizado.
- **Capacidad Predictiva**: Mientras que el suavizado ofrece una aproximación basada en componentes deterministas, el modelo **SARIMA** $(0,1,1)\times(1,0,1)_{12}$ logra un ajuste superior.

- **Fiabilidad Estadística**: La validez del pronóstico se confirma al observar que los valores reales del periodo de prueba se mantuvieron dentro de los intervalos de confianza (**UCL** y **LCL**) calculados al 95%.

En definitiva, la metodología **Box-Jenkins** proporciona una estructura más robusta para la previsión de la demanda turística, permitiendo una planificación de recursos más fiable gracias a la cuantificación de la incertidumbre mediante sus bandas de confianza, dándonos un análisis más preciso.